

Atelier 7 : le registre en classes

Par **Robert DIASSÉ**, Ingénieur Logiciel et Data

> **Cours** : Python avancé · **Chapitre** : 7, Introduction à la POO · **Version** : 2026-10-31

Avant de commencer

Ce que vous allez construire	Le paquet `yonnee` réécrit avec des classes : `Tarifs`, une `Livraison` (`dataclass`) qui **se vérifie elle-même**, un `Registre` qui **se charge, se filtre et se résume**, et une famille d'exceptions. Le programme principal doit redonner **exactement** les chiffres de l'atelier 6
Durée	🟢 Débutant : 7 h · 🔵 Standard : 5 h · 🟣 Expert : 3 h 30 (sans les défis)
Prérequis	Avoir lu le chapitre 7 ; l'atelier 6 terminé (paquet `yonnee`, environnement virtuel, `tabulate`)
Fichiers fournis	Aucun nouveau : les registres CSV des ateliers précédents (`registre_octobre.csv`, `registre_complet.csv`)
Rendu	Votre dépôt `yonnee-express`, avec l'étiquette `atelier-7`

Le résultat attendu, identique à celui de l'atelier 6 (seule la version change) :

```
Yonnee Express, outil version 2.0
Octobre : 53 livraisons valides, 7 rejetées
Registre complet : 2000 livraisons, 0 rejetées

Frais par quartier
| quartier   |   frais (F) |
|------------|-------------|
| Pikine     |     1076000 |
| Yoff       |      967000 |
| Parcelles  |      894000 |
| Plateau    |      729000 |
| Médina     |      506000 |

Durée moyenne par livreur
| livreur   |   minutes |
|-----------|-----------|
| Ibou      |      45.6 |
| Awa       |      45.1 |
| Fatou     |      45.0 |
| Modou     |      44.5 |
| Coumba    |      44.0 |

Retards à Pikine : 109
Colis de plus de 8 kg livrés par Awa en 2026 : 54
```

> 📍 **Point de départ : votre paquet `yonnee`** de l'atelier 6. Les modules `filtres.py`, `outils.py` et `rapports.py` changent peu ; `modele.py` et `lecture.py` sont remplacés par des classes.
>
> **La suite** : c'est la fin du cours de Python avancé. Le typage (`mypy`), les tests (`pytest`) et l'outillage professionnel viendront ensuite, comme prévu par le syllabus.

Comment travailler

L'atelier est découpé en **7 étapes** (0 à 6). Après chaque étape, lancez `python main.py` : les chiffres ne doivent **jamais** changer.

Trois règles pour tout l'atelier

1. **Une classe, une responsabilité** : les tarifs, une livraison, un registre.
2. **Une `Livraison` ne peut pas exister fausse** : toute vérification se fait à sa création.
3. **Une étape réussie = un commit.**

Étape 0 : la référence (10 min)

🎯 **Objectif** : savoir exactement ce que le programme doit afficher.

🛠️ **A faire** : lancez `python main.py` (version de l'atelier 6) et enregistrez la sortie : `python main.py > reference.txt` (le journal, lui, va à l'écran et dans `journal.log`).

✅ **Vérification** : `reference.txt` contient les chiffres de l'aperçu, avec la version 1.0.

Étape 1 : les exceptions (20 min)

🎯 **Objectif** : une famille d'erreurs propre à l'application.

🛠️ **A faire** : créez `yonnee/erreurs.py` avec `ErreurYonnee(Exception)` et `LivraisonInvalide(ErreurYonnee, ValueError)`.

🔎 **A chercher** : chapitre 7, section 8.

💡 **A comprendre**

- Pourquoi `LivraisonInvalide` hérite-t-elle **aussi** de `ValueError` ? (Pensez au code qui attrapait déjà `ValueError` à l'atelier 6.)

✅ **Vérification** : dans un terminal Python, `issubclass(LivraisonInvalide, ValueError)` et `issubclass(LivraisonInvalide, ErreurYonnee)` valent `True`.

Étape 2 : la classe `Tarifs` (40 min)

🎯 **Objectif** : les tarifs et le calcul des frais au même endroit.

🛠️ **A faire** : dans `yonnee/tarifs.py`, une classe `Tarifs` :

1. des attributs de classe `PAR_DEFAUT` (le dictionnaire des tarifs), `SUPPLEMENT_LOURD = 500`, `POIDS_LOURD = 5` ;
2. `__init__(self, prix_par_quartier=None)` : garde une **copie** des prix donnés, ou des prix par défaut, dans `self._prix` ;
3. une méthode de classe `depuis_fichier(cls, chemin)`, qui reprend `charger_tarifs` de l'atelier 6 (journal compris) et renvoie un objet `Tarifs` ;
4. `__contains__(self, quartier)`, pour pouvoir écrire `"Pikine" in tarifs` ;
5. `frais(self, quartier, poids)`.

🔎 **A chercher** : chapitre 7, sections 2, 4 et 10.

💡 **A comprendre**

- Pourquoi une **copie** (`dict(...)`) des prix ? Que se passerait-il si deux objets `Tarifs` partageaient le même dictionnaire et que l'un modifiait un prix ?
- Pourquoi `depuis_fichier` est-elle une méthode **de classe** ?

✅ **Vérification** : `Tarifs().frais("Pikine", 7.5)` vaut 3000 ; `Tarifs().frais("Médina", 2)` vaut 1000 ; `"Thiès" in Tarifs()` vaut `False`.

```
git add .
git commit -m "Ajoute les exceptions et la classe Tarifs"
```

Étape 3 : la `Livraison` qui se vérifie (60 min)

🎯 **Objectif** : impossible de créer une livraison fausse.

🛠️ **A faire** : dans `yonnee/livraison.py`, un `@dataclass` `Livraison` avec les huit champs de l'ancien tuple nommé (`numero: int`, `date: date`, `livreur: str`, `quartier: str`, `poids: float`, `montant: int`, `frais: int`, `duree_min: int`).

1. `__post_init__` lève `LivraisonInvalide` si le livreur est vide, si le poids n'est pas positif, ou si le montant sort de 500 à 500 000 F.
2. Une méthode de classe `depuis_ligne(cls, ligne, tarifs)` qui reprend `convertir` de l'atelier 6 : elle convertit les textes, vérifie le quartier et les frais, et **transforme** toute `ValueError` de conversion (`float("2,5")`, une date impossible) en `LivraisonInvalide` avec `raise LivraisonInvalide(str(e)) from e`.
3. Une méthode `est_en_retard(self, seuil=60)` et un `__str__` court.

🔎 **A chercher** : chapitre 7, sections 9 et 10 ; « Python raise from ».

💡 **A comprendre**

- Quelles vérifications vont dans `__post_init__`, et lesquelles dans `depuis_ligne` ? (Indice : lesquelles ont besoin des tarifs ?)
- Que signifie `from e` ? Qu'affiche la trace si l'on ne le met pas ?

✅ **Vérification** : `Livraison(1, date(2026, 10, 5), "Awa", "Pikine", 7.5, 25000, 3000, 75)` se crée, `est_en_retard()` vaut `True` et `est_en_retard(seuil=90)` vaut `False` ; avec un livreur vide, un poids de -1 ou un montant de 100, la création lève `LivraisonInvalide` (`livreur vide`, `poids invalide : -1`, `montant hors limites : 100`) ; une ligne de Yoff à 6 kg avec 2000 F de frais est refusée par `depuis_ligne` (`frais incohérents : 2000 au lieu de 2500`).

```
git add .
git commit -m "Ajoute la classe Livraison"
```

Étape 4 : le `Registre` (60 min)

🎯 **Objectif** : une collection de livraisons qui sait tout faire.

🛠️ **A faire** : dans `yonnee/registre.py`, une classe `Registre` :

1. `__init__(self, livraisons=None)` : une liste **interne** `self._livraisons`, et une liste `self.rejets` vide ;
2. une méthode de classe `charger(cls, chemin, tarifs)` qui reprend `lire_registre` : chaque ligne valide est ajoutée, chaque `LivraisonInvalide` est journalisée et gardée dans `rejets` (numéro de ligne, message) ;
3. `ajouter`, `__len__` et `__iter__` (pour écrire `for l in registre:` et `len(registre)`) ;
4. `filtrer(self, *conditions)`, qui renvoie un **nouveau** `Registre` ;
5. `rapport(self, cle, valeur=..., agregat=sum)`, qui reprend la fonction `rapport` de l'atelier 4.

Dans `rapports.py`, il ne reste que `moyenne` et `en_tableau`.

🔎 **A chercher** : chapitre 7, sections 4 et 5 ; « Python **iter** ».

💡 **A comprendre**

- Pourquoi `filtrer` renvoie-t-il un `Registre`, et pas une liste ? (Que peut-on faire ensuite sur le résultat ?)
- Avec `__iter__`, la fonction `filtrer` de l'atelier 4 (`filtres.py`) marche-t-elle encore sur un registre ? Pourquoi ?

✅ **Vérification** : un registre de deux livraisons (Awa à Pikine, Modou à Yoff) : `len(r)` vaut 2, `len(r.filtrer(lambda x: x.quartier == "Yoff"))` vaut 1, et `r.rapport(cle=lambda x: x.livreur)` vaut `{'Awa': 3000, 'Modou': 2000}`.

```
git add .
git commit -m "Ajoute la classe Registre"
```

Étape 5 : le programme principal (30 min)

🎯 **Objectif** : retrouver la sortie de référence.

🛠️ **A faire**

1. Réécrivez `main.py` avec les classes : `Tarifs.depuis_fichier(...)`, `Registre.charger(...)`, `registre.rapport(...)`, `registre.filtrer(...)`, `len(...)` ; la version passe à `2.0`.
2. Supprimez `modele.py` et `lecture.py` (tout est maintenant dans les classes).
3. Comparez avec `reference.txt` (VS Code : clic droit sur un fichier, **Select for Compare**, puis **Compare with Selected**).

✅ **Vérification** : seule la ligne de version diffère ; les 7 rejets d'octobre sont les mêmes (lignes 6, 13, 19, 25, 32, 40, 47), seul le nom du journal passe de `yonnee.lecture` à `yonnee.registre`.

```
git add .
git commit -m "Réécrit l'outil avec des classes"
```

Étape 6 : le bilan et l'étiquette (20 min)

🎯 **Objectif** : mesurer ce que la POO a changé.

🛠️ **A faire**

1. Dans le `README.md` du dépôt, répondez :
   - Les trois limites du chapitre 6 (section 8) sont-elles levées ? Montrez, pour chacune, la ligne de code qui le prouve.
   - Pour ajouter un quartier, ou une sorte de livraison « express » au tarif double, quelles classes faut-il modifier ou créer ?
2. Étiquetez :

```
git tag atelier-7
git push
git push origin atelier-7
```

✅ **Vérification** : sur GitHub, l'étiquette `atelier-7`.

🏆 Défis (facultatifs)

1. **La livraison express.** Une classe `LivraisonExpress(Livraison)` dont les frais attendus sont doublés ; adaptez `depuis_ligne` (une colonne `express` dans le CSV). Le reste du programme ne doit pas changer.
2. **Le livreur.** Une classe `Livreur` (nom, véhicule) et une méthode `Registre.par_livreur()` qui renvoie un dictionnaire nom → `Registre`.
3. **Comparer des registres.** `__eq__` et `__add__` pour `Registre` : `octobre + novembre` renvoie un registre qui contient les deux.
4. **Les types.** Annotez toutes vos méthodes (`def frais(self, quartier: str, poids: float) -> int:`), installez `mypy` dans l'environnement virtuel et lancez `mypy yonnee` : corrigez ce qu'il signale.

Grille d'autoévaluation

Je sais…	Oui	Pas encore
écrire une classe avec `__init__`, des attributs et des méthodes	☐	☐
utiliser une méthode de classe comme autre constructeur	☐	☐
empêcher la création d'un objet faux (`__post_init__`, exceptions)	☐	☐
créer une famille d'exceptions et transformer une erreur avec `raise ... from`	☐	☐
rendre un objet utilisable avec `len`, `for` et `in`	☐	☐

Je sais…	Oui	Pas encore
réécrire un programme sans changer ses résultats, en comparant la sortie	☐	☐